

Voice RAG Assistant

System Architecture & Technical Documentation

Document version 2.0

1. System Overview

The Voice RAG Assistant is a full-stack retrieval-augmented generation (RAG) application that lets users ask questions about enterprise policy documents using voice. The system ingests PDF documents, splits them into chunks, embeds them into a vector store, and answers questions by retrieving the most relevant chunks and synthesising an answer with a large language model (DeepSeek by default, with a Gemini fallback chain). Answers are delivered as streaming text (SSE) and spoken aloud via a text-to-speech service.

The application is split into two independently deployable parts:

- Backend - Python / FastAPI, at `http://localhost:8000`
- Frontend - React / TypeScript / Vite, at `http://localhost:5173`

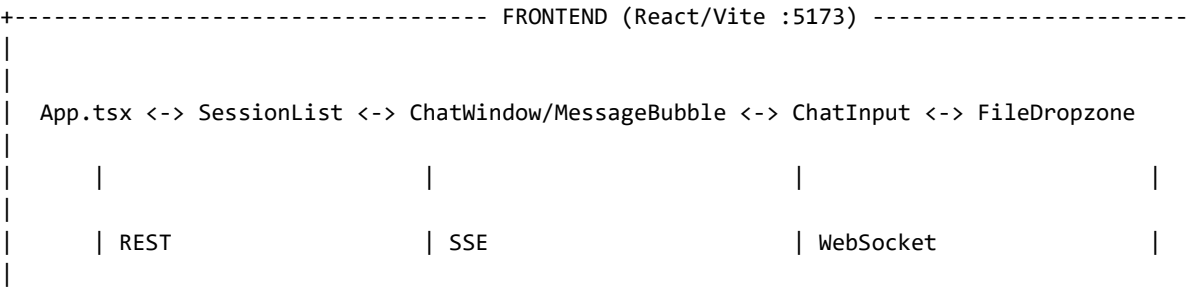
Communication between the two is performed over HTTP (REST + Server-Sent Events) and a WebSocket used for real-time TTS audio.

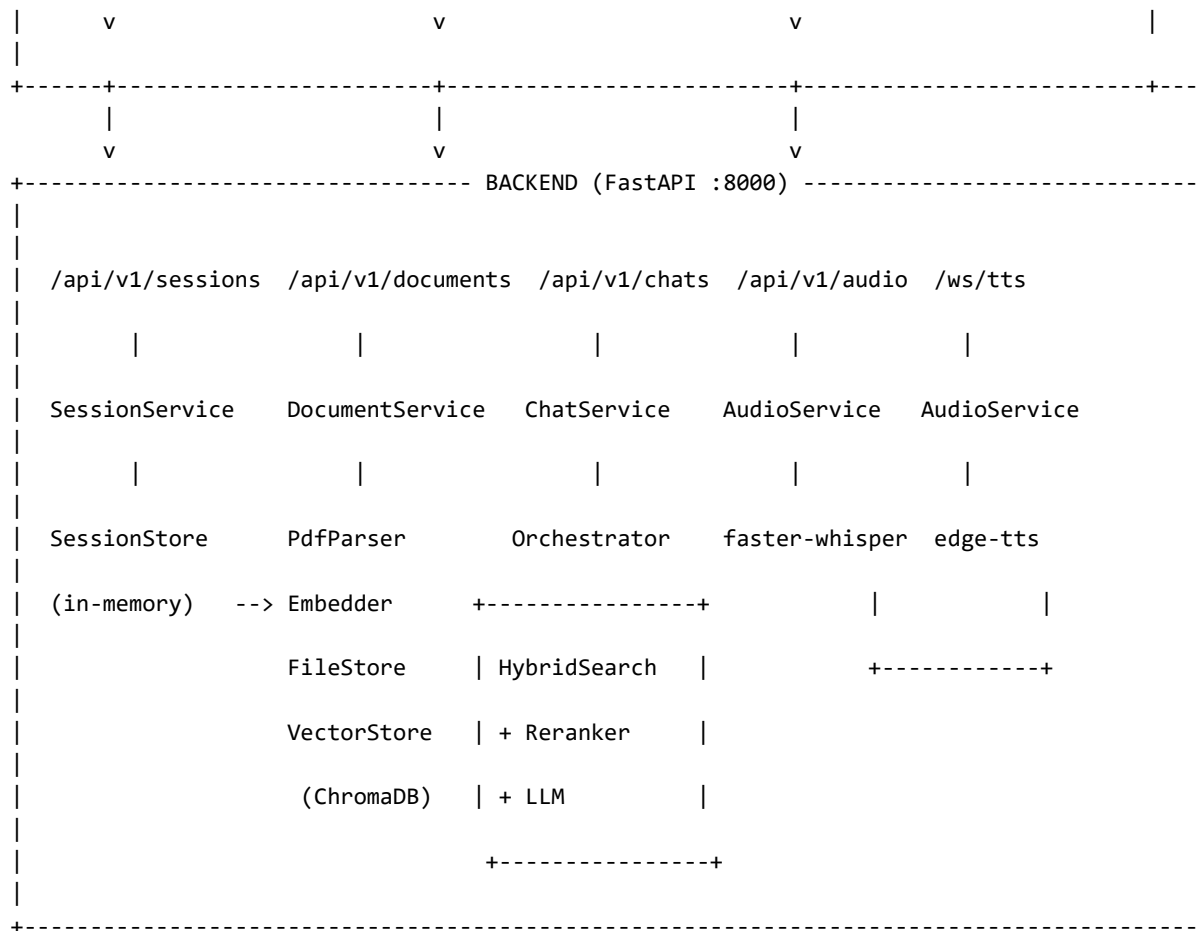
Technology stack

Layer	Technology
Backend framework	Python 3, FastAPI, Uvicorn
Frontend framework	React 18, TypeScript, Vite, Tailwind CSS
Embedding model	BAAI/bge-m3 (SentenceTransformer)
Reranker	BAAI/bge-reranker-v2-m3
LLM	DeepSeek (deepseek-chat) - primary; Gemini fallback
Vector store	ChromaDB (default) or FAISS
Lexical search	BM25Okapi (rank_bm25)
PDF parsing	PyMuPDF (fitz) + recursive & semantic splitters
ASR (speech-to-text)	faster-whisper (Whisper base)
TTS (text-to-speech)	edge-tts (Microsoft neural voices)
Networking	REST, SSE, WebSocket

2. High-Level Architecture

The diagram below shows the main subsystems and the data flow between them.





3. Core Request Flows

3.1 Document Upload & Indexing

1. User uploads a PDF via FileDropzone; the frontend calls POST `/api/v1/documents/upload`.
2. DocumentService saves the file via FileStore and parses it with PdfParser (default semantic chunking, configurable recursive chunking).
3. Chapter detection (regex) and keyword-based chapter assignment tag each chunk with metadata: doc_id, page, chunk_index, chapter.
4. Chunks are embedded in batches of 32 by the Embedder (BAAI/bge-m3) and inserted into the vector store.
5. Embedding runs in a background thread; the frontend subscribes to GET `/api/v1/documents/{id}/progress` (SSE) to show upload/embedding progress.

3.2 Question Answering (Streaming)

1. User taps the microphone; the frontend records WebM audio and calls POST `/api/v1/audio/transcribe`.
2. faster-whisper transcribes the audio to text (offloaded with `asyncio.to_thread`).
3. The frontend opens POST `/api/v1/chats/query/stream` (SSE) with the transcript.

4. ChatService appends the user message, builds conversation history, and streams the answer through the pipeline (see section 4).
5. Tokens arrive as SSE events and are rendered live in the chat window. On completion, a 'done' event delivers the retrieved source chunks (citations).
6. Meanwhile, the frontend flushes completed sentences to the TTS WebSocket so audio starts speaking before the full answer has been generated.

4. Retrieval-Augmented Generation Pipeline

The retrieval pipeline is layered for robustness. The streaming path used in production is `PipelineOrchestrator.answer_question_stream()`. The non-streaming path uses `answer_question()`, which also runs the same direct retrieval. Note: a multi-stage fallback pipeline (query expansion, HyDE, agent classification, SelfRAG) is implemented but is currently inactive implementation - none of its entry points (`_full_path`, `fallback.execute`, `agent`, `hyde`) are invoked by any active code path.

4.1 Streaming path (production)

Stage	Component	What it does
1. Cache check	QueryCache	Returns a cached answer for repeated queries (TTL 1h, max 100).
2. Hybrid retrieval	HybridSearch	Combines vector search (bge-m3) with BM25 lexical search using Reciprocal Rank Fusion.
3. Reranking	Reranker	Cross-encoder re-scores the top candidates (top_k=5).
4. Generation	LLMClient	Streams tokens from the configured provider - DeepSeek (deepseek-chat) by default, falling back to Gemini (gemini-2.0-flash-lite) - with a policy-assistant system prompt.

4.2 Query normalisation (HybridSearch)

- Filler words removed (um, uh, like, actually, ...).
- Abbreviations expanded to full forms (e.g. SSO -> single sign-on).
- URLs replaced with a [URL] placeholder.
- Metadata hints extracted: error codes (e.g. ERR-SSO-4039), form codes (e.g. HR-PAY-102), and chapter filters.

4.3 Fallback pipeline (NOT wired in)

`PipelineOrchestrator` defines `_full_path()` and `_handle_out_of_scope()`, and `main.py` constructs `Fallback`, `AgentRAG`, `QueryExpander` and `HyDE`. However, neither `answer_question()` nor `answer_question_stream()` calls them - `answer_question()` calls `_simple_path()` directly. The fallback chain below is currently inactive implementation kept in the repository, not part of the running system:

Priority	Strategy	Notes
1	Hybrid + reranker	Direct retrieval then rerank.
2	Query expansion	Gemini expands/simplifies the query, then retries stage 1.
3	HyDE	Gemini writes a hypothetical answer, uses it for retrieval, then reranks.

AgentRAG.classify() labels query types (simple_fact, complex_reasoning, comparison, summarization, out_of_scope) and would be used by the fallback to skip out-of-scope queries. SelfRAG (relevance verification) is likewise defined but not integrated into any active code path.

5. Voice Pipeline (ASR + TTS)

5.1 Speech-to-text (ASR)

Audio is recorded in the browser (WebM/Opus, 16 kHz mono) with echo cancellation, noise suppression and auto gain control. The AudioService uses faster-whisper (Whisper 'base' model) to transcribe.

Feature	Detail
Model	faster-whisper base
Device	CUDA when available, else CPU with int8
Endpoint	POST /api/v1/audio/transcribe
Output	Transcript text + language + duration

5.2 Text-to-speech (TTS)

Two transports exist:

- WebSocket /ws/tts - one socket per sentence segment; MP3 audio chunks stream back as binary frames, terminated by a JSON {'status': 'done'} message. Used by the chat UI for live speaking.
- REST - POST /api/v1/audio/synthesize and /synthesize-stream return MP3 audio for ad-hoc playback.

Sentence-boundary flushing (frontend App.tsx): as answer tokens stream in, text is accumulated; once a sentence boundary (. ! ? 。 ! ? or newline) is detected, the completed sentence is immediately sent to the TTS engine. This allows speech playback to begin before the entire response has been generated. The flushTts() function continuously processes completed sentences (up to MAX_SEGMENT = 350 characters per segment), while a 400 ms debounce timer force-flushes any remaining text to avoid excessive delays for long responses without punctuation.

Config	Default	Description
tts_voice	en-US-AndrewMultilingualNeural	Edge-tts neural voice
tts_rate	+0%	Speaking rate offset

tts_pitch	+0Hz	Pitch offset
------------------	------	--------------

Before synthesis, `clean_text_for_tts()` strips markdown characters, replaces slashes with spaces, and normalises dashes (em/en dashes and spaced hyphens become commas) to avoid unnatural long pauses; hyphens inside words (e.g. HR-PAY-102) are preserved. Synthesised audio is cached (TTSCache, max 100) keyed by MD5 of the cleaned text.

6. Data Model & Storage

6.1 Domain model (app/models/domain.py)

Entity	Fields
Session	session_id (uuid), created_at, title, messages[]
Message	role (user/assistant), content, timestamp
Document	doc_id, filename, file_path, uploaded_at
Chunk	doc_id, page, chunk_index, text, embedding

6.2 Stores

Store	Persistence	Notes
SessionStore	In-memory	Sessions & messages lost on restart (known limitation).
FileStore	Disk (uploads/ + documents.json)	Original PDFs and metadata.
VectorStore (ChromaDB)	Disk (chroma_db/)	Default backend; cosine space.
VectorStore (FAISS)	Disk (faiss_index/)	Alternative backend, switchable via config.
QueryCache	In-memory	Answer cache, TTL 1h, max 100.
TTSCache	In-memory	Synthesised audio, max 100.

6.3 Chunk metadata

- doc_id - owning document
- page - source PDF page number
- chunk_index - position within the document
- chapter - detected chapter (e.g. 'Chapter 3: ...') or null
- text - chunk content

7. API Reference

All REST endpoints are prefixed `/api/v1` and proxied by Vite in development.

7.1 Sessions

Method	Path	Purpose
POST	/sessions	Create a new session
GET	/sessions	List sessions
PATCH	/sessions/{id}	Rename a session (Form: title)
DELETE	/sessions/{id}	Delete a session

7.2 Documents

Method	Path	Purpose
POST	/documents/upload	Upload a PDF (max 40 MB)
GET	/documents/	List uploaded documents
GET	/documents/{id}/progress	SSE embedding progress
DELETE	/documents/{id}	Delete a document

7.3 Chats

Method	Path	Purpose
POST	/chats/query	Non-streaming answer (returns chunks too)
POST	/chats/query/stream	SSE streaming answer
GET	/chats/{id}/messages	Fetch conversation history
POST	/chats/clear-cache	Clear the query cache

7.4 Audio

Method	Path	Purpose
POST	/audio/transcribe	Transcribe uploaded audio
POST	/audio/synthesize	Synthesise speech to MP3
POST	/audio/synthesize-stream	Stream MP3 audio
WS	/ws/tts	WebSocket TTS for live chat audio

8. Frontend Architecture

The frontend is a React + TypeScript single-page app styled with Tailwind CSS.

8.1 Component tree

```
App.tsx
+-- AppShell (layout: collapsible sidebar + main area)
|   +-- Header (main area top bar)
|   +-- sidebar sections:
|       +-- SessionList (chats, rename/delete)
|       +-- FileDropzone + UploadProgress
|       +-- FileList (documents)
+-- ChatWindow
|   +-- MessageBubble (user/assistant, sources)
|   +-- TypingIndicator
+-- ChatInput
    +-- RecordButton (mic / recording / transcribing)
    +-- AudioVisualizer (live level bars)
```

8.2 State & hooks

Hook	Responsibility
App.tsx (local state)	Sessions, current chat, messages, documents, upload state, TTS streaming.
useTtsSocket	WebSocket lifecycle, audio queue, sequential playback.
useAudioAnalyser	Web Audio API analyser for the live visualiser.
useAudioRecorder	MediaRecorder wrapper (defined, not used by ChatInput).
useSessions	Session CRUD wrapper (defined, not wired in App).
useAudioPlayer	Audio playback helper (defined, unused).

8.3 Streaming UX

- Text streams via SSE and re-renders per token; the chat auto-scrolls only when the message count grows.
- Assistant answers are card-style with collapsible 'Show sources' listing page number, match % and snippet.
- The mic button doubles as recording/processing indicator; a recording bar shows an elapsed timer and live visualiser.

9. Configuration

Backend settings live in app/config.py (pydantic-settings), overridable via a .env file.

Setting	Default	Purpose
llm_provider	deepseek	LLM provider: "gemini", "deepseek" or "auto".
gemini_api_key	(unset)	Gemini API key used when provider is gemini/auto or for fallback/agent/hyde/expander.

deepseek_api_key	(unset)	DeepSeek API key (falls back to gemini_api_key if blank).
deepseek_base_url	https://api.deepseek.com	DeepSeek OpenAI-compatible base URL.
deepseek_model_name	deepseek-chat	DeepSeek model name.
host / port	0.0.0.0 / 8000	Backend bind address.
cors_origins	http://localhost:5173	Allowed CORS origins (comma-separated).
whisper_model_size	base	faster-whisper model.
whisper_use_gpu	true	Auto-falls back to CPU when CUDA unavailable.
embedding_model_name	BAAI/bge-m3	Embedding model.
reranker_model_name	BAAI/bge-reranker-v2-m3	Cross-encoder.
vector_store_type	chroma	chroma or faiss.
chunk_size / overlap	512 / 64	Recursive chunking parameters.
retrieval_top_k	10	Retrieval candidate count.
retrieval_fetch_k	15	Chunks fetched from the vector store before filtering.
reranker_top_k	5	Chunks passed to the LLM.
upload_dir	uploads	PDF storage directory.
max_file_size_mb	40	Upload limit.

LLM rate limiting

An AsyncRateLimiter caps Gemini fallback calls at 15 requests/minute to avoid 429 errors. Callers (LLM, QueryExpander, AgentRAG, HyDE) retry with backoff on RESOURCE_EXHAUSTED. DeepSeek is the default provider and is tried first; Gemini is only used when llm_provider is "gemini"/"auto" or when DeepSeek fails.

10. Testing

Backend tests live under backend/tests/ and exercise the pipeline and services:

File	Focus
test_asr.py	Whisper transcription round-trip.
test_latency.py	Per-query latency, including TTS synthesis timing.
test_load.py	Load/throughput checks.
test_vector_search.py	Vector store retrieval correctness.
evaluate_rag.py	RAG quality evaluation.

evaluate_ragas.py	Ragas-based metric evaluation.
run_tests.py	Test runner.
voice_test_dataset.json	10-question latency test dataset.

11. Deployment & Operations

Run locally

```
# Backend
cd backend
uvicorn app.main:app --reload --port 8000

# Frontend
cd frontend
npm run dev    # Vite on :5173, proxies /api and /ws to :8000
```

Production notes

- Vite dev proxy forwards /api and /ws to the backend; production must replicate this (reverse proxy or shared origin).
- Stores are in-memory for sessions and disk-based for documents/vectors; sessions do not survive a restart.
- GPU is auto-detected for Whisper, embedding and reranking; the system runs on CPU if CUDA is absent.
- DeepSeek is the default LLM; the Gemini free tier (15 requests/minute, enforced by the rate limiter) is a fallback.
- A Dockerfile exists for the frontend (container build is optional).

12. Known Limitations

- Sessions and messages are in-memory only; they are lost when the backend restarts.
- The multi-stage fallback pipeline (query expansion, HyDE, agent classification, SelfRAG) is implemented but currently inactive implementation - no active path (streaming or non-streaming) invokes it. Only the direct hybrid retrieval + rerank + LLM path runs.
- TTS uses the free edge-tts service; each sentence segment requires a new WebSocket connection.
- The tts.py and asr.py pipeline stubs are placeholders; real ASR/TTS lives in AudioService.